

Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs

Fangkai Yang
Microsoft Research
China

Lu Wang
Microsoft Research
China

Zhenyu Xu
Microsoft Research
China

Jue Zhang
Microsoft Research
China

Liqun Li
Microsoft Research
China

Bo Qiao
Microsoft Research
China

Camille Couturier
Microsoft 365
France

Chetan Bansal
Microsoft 365
USA

Soumya Ram
Microsoft Azure
USA

Si Qin
Microsoft Research
China

Zhen Ma
Microsoft 365
USA

Íñigo Goiri
Microsoft Azure
USA

Eli Cortez
Microsoft Azure
USA

Terry Yang
Microsoft 365
China

Victor Rühle
Microsoft 365
UK

Saravan Rajmohan
Microsoft 365
USA

Qingwei Lin*
Microsoft Research
China

Dongmei Zhang
Microsoft Research
China

ABSTRACT

Cloud providers often have resources that are not being fully utilized, and they may offer them at a lower cost to make up for the reduced availability of these resources. However, customers may be hesitant to use such offerings (such as spot VMs) as making trade-offs between cost and resource availability is not always straightforward. In this work, we propose Snape (Spot On-demand Perfect Mixture), an intelligent framework to optimize the cost and resource availability by dynamically mixing on-demand VMs with spot VMs. Through a detailed characterization based on real production traces, we verify that the eviction of spot VMs is predictable to some extent. Snape also leverages constrained reinforcement learning to adjust the mixture policy online. Experiments across different configurations show that Snape achieves 44% savings compared to using only on-demand VMs while maintaining 99.96% availability, which is 2.77% higher than using only spot VMs.

CCS CONCEPTS

• **Computer systems organization** → **Cloud computing**; • **Theory of computation** → **Scheduling algorithms**; • **Computing methodologies** → **Reinforcement learning**.

KEYWORDS

Spot virtual machine, eviction prediction, dynamic mixture

*Qingwei Lin is the corresponding author of this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASPLOS '23, March 25–29, 2023, Vancouver, BC, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9918-0/23/03...\$15.00

<https://doi.org/10.1145/3582016.3582028>

ACM Reference Format:

Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liqun Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23), March 25–29, 2023, Vancouver, BC, Canada*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3582016.3582028>

1 INTRODUCTION

Large cloud providers typically ensure that their services are always available and reliable by allocating a sufficient amount of computing resources, so that it appears to customers that resources are never in short supply. This often leads to underutilized resources, and cloud providers are motivated to monetize excess capacity by selling them at discounted prices to compensate for the degradation in resource availability. Example offerings include spot virtual machine (VM) [2, 11, 29], spot block VM [10] and harvest VM [3, 26, 57].

The trade-off between cost-effectiveness and resource availability often creates a dilemma for users. On the one hand, some workloads, such as machine learning training and inferencing, caching, and big data processing, can potentially run at reduced reliability for lower cost. On the other hand, cloud providers, while sharing some information about resource availability such as estimated eviction rates for spot VMs at the region level, do not guarantee resource availability. Figure 1 shows an incident caused by the unexpected high evictions of spot VMs. For months, prior to the incident, the eviction rate had remained stable, leading service owners to believe it would be safe to use spot VMs with some level of overprovisioning based on the estimated eviction rate. However, around 4am, the eviction rate suddenly increased, resulting in the eviction of approximately 60% of spot VMs. Although there was no noticeable impact at the time due to low traffic, the service health began to drop around 3pm as traffic increased.

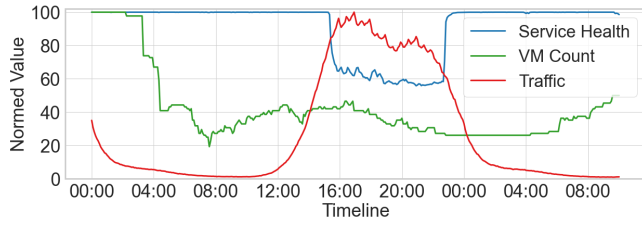


Figure 1: A real service incident due to unexpected high eviction rates of spot VMs (metrics are normalized and time is shifted to UTC).

One natural solution to prevent similar incidents as the above is to deploy services in a VM Scaling Group (VMSG) with both spot and on-demand VMs, so that there would still be enough on-demand VMs to support services even if all spot instances were evicted. Cloud providers have begun providing such offerings recently [7, 16] as described in Section 2.1. However, all of them either support a static mixture ratio of spot and on-demand VMs or allocate spot instances in a greedy fashion, which can be less cost-effective when the eviction rate is low. To better balance cost and resource reliability, this work proposes an intelligent framework Snape (**S**pot **O**n-demand **P**erfect **M**ixture), which dynamically adjusts the number of on-demand and spot VMs by leveraging the fact that the eviction of spot VMs is predictable to some extent.

As the first step to demonstrate the effectiveness of Snape, we introduce a new VMSG in which the number of in-service VMs is equal to or greater than a specified target number that meets a service level objective (SLO), while allowing the number of spot and on-demand VMs to vary for optimized cost saving. We defined the VMSG SLO in Section 3.2 regarding the availability of VMs. Customers provide both the target number of in-service VMs and the Service Level Objectives (SLOs). SLOs can be less strict (e.g., three nines) as than those for on-demand VMs (four or five nines [9, 13]). Obtaining an optimal mixture plan is challenging on several aspects:

- (1) To predict the evictions of spot VMs, the various factors leading to eviction at both global and local levels need to be studied [58].
- (2) The uncertainty associated with eviction rate prediction makes tracking the SLO over time complex; optimal mixture decisions would depend on the available SLO budget and the level of uncertainty in the prediction.
- (3) A less-optimized “myopic” decision could be made if only short-term eviction prediction results are considered. This can arise when the eviction rate is fast-varying.

To address these challenges, we first characterize the eviction behaviors of spot VMs by examining traces collected over three months from a leading cloud provider in Section 3. For example, we find that the eviction rate is higher on weekdays and working hours, and spot VMs are more likely to get evicted if colocated with a few on-demand VMs. Furthermore, to better learn the eviction behaviors and achieve optimal decisions in the long term, we employ the framework of constrained reinforcement learning (CRL) and discuss how CRL can be applied to the current problem. The

overall architecture of Snape and its components are introduced in Section 4, followed by a detailed evaluation in Section 5.

Our contributions can be summarized as follows:

- To the best of our knowledge, we are the first to conduct a detailed characterization of the eviction behavior of spot VMs by analyzing real production traces from Microsoft Azure.
- We design a constrained reinforcement learning framework, Snape, which can dynamically adjust the mixture number of on-demand and spot VM to maximize cost savings as well as satisfy the specified SLO provided by users.
- We implement an example VM scaling group under the framework of Snape. We validate its effectiveness in experiments that Snape achieves 44% savings than on-demand VMSG while retaining 99.96% availability.

2 BACKGROUND

2.1 VM Scaling Groups

VM Scaling Group (VMSG) has been offered by all major cloud providers though under different names, namely, AWS ASG [7], GCP MIG [22], and Azure VMSS [17]. A scaling group consists of a group of VMs managed by an autoscaler based on a specific rule (e.g., a predefined schedule) or target metrics such as CPU utilization, request per second, or predicted workload.

In its early days, a VMSG was typically composed of homogeneous VMs, i.e., with the same SKU size and in the same mode (spot or on-demand). Recently, clouds (e.g., Azure and AWS) introduced mixed VMSGs which can flexibly orchestrate VMs of different sizes and modes [7, 16]. The goal is to benefit from the discount brought by spot instances while fulfilling the basic resource requirement with the on-demand instances. The user is required to configure the number of minimum on-demand instances and the mixture ratio. The autoscaler is responsible for maintaining the proportion of each VM type. For example, if the minimum number of on-demand instances is 4 and the mixture ratio is 0.5, then when the scaling group size is 10, there are 7 on-demand instances plus 3 spot instances. Detailed orchestration behaviors for AWS can be found here [7]. For spot instance evictions, we notice that the auto-restoration is supported [7, 17, 22], which means that the autoscaler will automatically restart the evicted spot instances if capacity permits. However, this is not guaranteed especially during peak hours due to constrained capacity. We studied the spot VM auto-restoration which will be detailed in Section 3.2.

Third-party orchestrators also exist such as AutoSpotting [6] which greedily allocates spot instances to fulfill the required number of VMs. When spot instances get evicted, it immediately allocates on-demand instances to ensure the required capacity. Then, when the spot instances are restored, AutoSpotting will shut down the on-demand instances. There is no explicit SLO for VMSG and only instance-level SLO for on-demand VMs now. We thus define the SLO of a VMSG as an extension of the instance-level SLO, which will be discussed in Section 3.2.

2.2 Constrained Reinforcement Learning

Constrained Reinforcement Learning (CRL) is a kind of Reinforcement Learning [53] algorithm whose goal is not only to maximize

long-term cumulative reward but also to satisfy the additional constraints without any heuristics or rules [27, 41]. CRL is a key field for real-world RL problems as ensuring the satisfaction of certain constraints is necessary and essential in practice. has seen great success in a variety of industries, including electric grids [36], networking [33], robotics [28, 47], resource allocation [18, 39], and wireless security [42]. Applying CRL to our problem, the objective is to maintain the number of in-service VMs, not dropping below the target number while minimizing the cost of using VMs in the long term. It is challenging to make such long-term decisions to maintain the target due to the complexity of the system, *e.g.*, the behavior of spot evictions. For instance, adding extra spot VMs to VMSG does not guarantee the target number but results in additional costs; conversely, always keeping a certain number of on-demand VMs is unnecessary (as discussed further in Section 5).

3 CHARACTERISTICS OF SPOT VMS AND EVICTIONS

In this section, we conduct empirical studies to analyze the behaviors of evictions concerning spot VMs and VMSG. To study spot VM evictions, we analyze the sampled traces of the eastern US region from Microsoft Azure. The trace includes spot VMs existed between April 15th 2022 and July 15th 2022. Note that different regions might exhibit widely different spot eviction behaviors. For the figures with no scale on the y-axis, the range might not be between 0% and 100%.

3.1 Spot VM Evictions

Eviction reason. Among all spot VMs in the trace, 99.96% spot VMs are terminated, including 6.05% ‘Evicted’ spot VMs and 93.91% ‘Deleted’ spot VMs (*i.e.*, users shut down spot VMs). It indicates that the majority of spot VMs are deleted, *e.g.*, scheduled workloads on spot VMs are completed. Only ‘Evicted’ spot VMs are considered in the below empirical studies. Evictions result from diverse reasons, including deployments of on-demand VMs, service healing, testing, etc. 90.79% evicted spot VMs are caused by the on-demand VM deployments, *i.e.*, the cloud platform needs the capacity back for on-demand VM deployments. It implies that the status of on-demand VMs could be valuable in predicting spot VM evictions.

Lifetime. Figure 2(a) shows the lifetime distribution of these spot VMs. 55.31% of the evicted spot VMs and 32.80% of the deleted spot VMs live longer than one hour. In addition, spot VMs have different types [8, 14] that have diverse characteristics for resources (*i.e.*, core-to-memory ratio, disk throughput, etc.) to fulfill different purposes. It is common that VM allocators of cloud providers tightly pack VMs by a single dominant resource (*i.e.*, cores) to simplify capacity management [3, 31, 56]. Thus, we group ‘Evicted’ spot VMs by cores to investigate their lifetime. Figure 2(b) shows their lifetime grouped by different cores. Spot VMs with larger cores have a short lifetime in general, which matches our expectations. When on-demand VMs need capacity back, nodes with larger spot VMs are more likely to be tightly packed with less remaining empty capacity, resulting in evictions.

Node status. The cloud system contains a large number of physical servers, *i.e.*, *Nodes*, that are allocated in different *Regions*. Each region contains many *Clusters* and each cluster contains hundreds

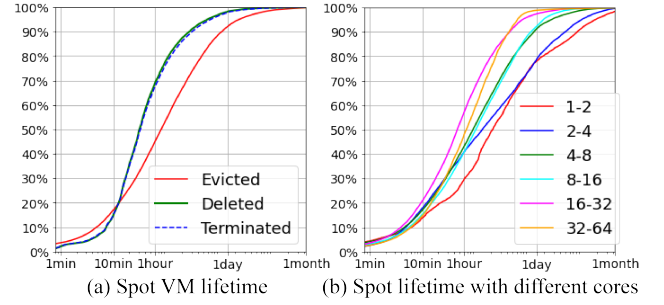


Figure 2: Distribution of the spot VM lifetime (log-scaled).

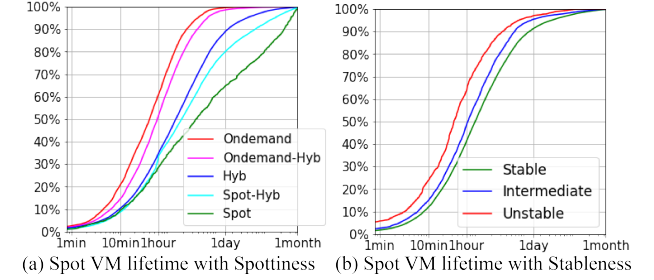


Figure 3: Distribution of the spot VM lifetime (log-scaled) on different node status.

of nodes [31]. VM allocator places spot and on-demand VMs on nodes, and node status affects the lifetime of spot VMs, *e.g.*, spot VMs in tightly packed nodes are more likely to be evicted leading to a shorter lifetime. In this paper, we define two metrics, *spottiness* and *stablness*, to evaluate the node status. *Spottiness* is defined as the difference between spot core percentage and on-demand core percentage of used cores in nodes (including spot and on-demand cores), *i.e.*, $\frac{Core_{spot}}{Core_{used}} - \frac{Core_{ondemand}}{Core_{used}}$. For each node, we take the average of spottiness over the evaluation time range and classify nodes by spottiness (*i.e.*, ‘Spot’: [0.98,1], ‘Spot-Hybrid’: [0.5,0.98], ‘Hybrid’: [-0.5,0.5], ‘Ondemand-Hybrid’: [-0.98,-0.5], ‘Ondemand’: [-1,-0.98]). Figure 3(a) shows that spot VMs on nodes with higher spottiness have longer lifetimes. It indicates that nodes packed with more on-demand cores are more likely to evict spot VMs on them, and nodes packed with more spot cores or even pure spot cores can host spot VMs for a longer time. On the other hand, *stablness* is defined as the variations of remaining core over-time, *i.e.*, $1 - \sum_{t=0}^{T-1} \text{abs}(\frac{Core_{remaining}^{t+1}}{Core_{Node}} - \frac{Core_{remaining}^t}{Core_{Node}}) / T$, where the remaining core is the node core capacity excluding occupied on-demand cores. We classify these nodes to *Stable*, *Intermediate*, and *Unstable* based on stableness with value in [0.6, 1], [0.4, 0.6) and [0, 0.4). Figure 3(b) indicates that spot VMs placed on nodes with more core variations will be evicted earlier. The remaining cores on these nodes are frequently picked to deploy new on-demand VMs, and spot VMs taking the remaining cores are more likely to be evicted.

Region status. Moving up to the region level, we analyze the allocated resources and new deployments to examine the region status. As aforementioned, the dominant eviction reason is the on-demand VM deployments. Figure 4 shows the hourly and weekly pattern of

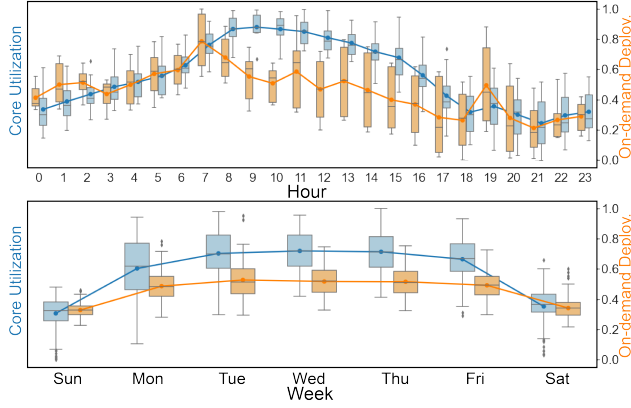


Figure 4: Hourly and weekly pattern of the core utilization (left y-axis) and on-demand VM deployments (right y-axis).

allocated resources and new on-demand deployments in order to connect with eviction patterns in the next section. The core utilization (left y-axis) is computed based on the cores of the currently running on-demand VMs over the whole capacity, *i.e.*, excluding resources that have been allocated to existing evictable VMs. The on-demand deployments (right y-axis) are normalized, representing the dynamics of on-demand VM creations over time. Note that we hide the y-axis for confidentiality reasons. The core utilization is higher during working hours and on weekdays, and the on-demand VM deployments behave in a similar higher pattern on weekdays. We observe a peak deployment around 7:00, followed by a rather lower number of deployments with larger variances. On-demand VMs created at this time survive during working hours, leading to fewer creations afterward. The large variances come from diverse requirements, *e.g.*, some services need additional deployments to host extra traffic, and some services have stable traffic during working hours without new deployments. Note that the small peak around 19:00 is caused by regular on-demand deployments at default time (UTC 0:00) shifted to the local time. The hourly pattern figure uses only the weekday data to reduce variances.

Eviction rate. The spot VM lifetime empirical study provides an overview of spot VM evictions within a future time span, and we analyze the spot eviction rate to investigate further the temporal pattern of evictions and its relations with other temporal factors (*e.g.*, core utilization and on-demand deployments in the previous section). We define the eviction rate as the percentage of the existing spot VMs being evicted within a given time period, *e.g.*, ‘1-hour’ eviction rate is computed by dividing the number of evicted spot VMs within 1 hour over the corresponding currently existing number of spot VMs. Figure 5 shows the eviction rate with different time horizons (*i.e.*, 1-, 2-, 3-, 6-, 12-, 24-hour) over time. The eviction rate is higher with a longer time horizon, and eviction rates are higher during working hours and on weekdays. In particular, the hourly patterns of eviction rates with short time horizons (1, 2, and 3 hours) follow the hourly patterns of allocated resources and on-demand deployments (Figure 4), showing higher eviction rates during working hours (between 07:00 to 17:00). On the other hand, the hourly patterns of eviction rates with long time horizons (6, 12, and 24 hours) show higher eviction rates around the start of

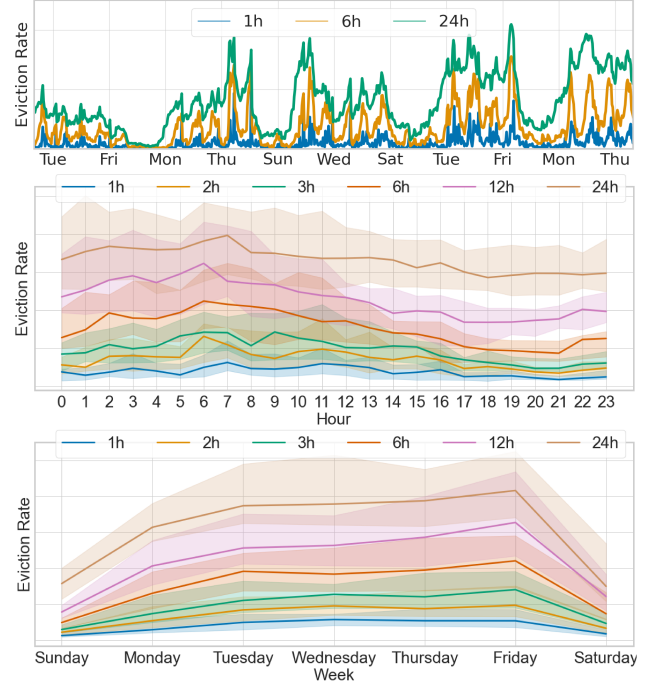


Figure 5: Weekly and hourly pattern of eviction rate with time horizons. The band represents the 25th and 75th percentiles. The eviction rates are hidden for confidentiality.

working hours, capturing evictions within longer time range that covers working hours.

3.2 VMSG Characteristics

Virtual Machine Scaling Group (VMSG) manages a group of VMs that application workloads are automatically distributed across them through a load balancer.

VMSG SLO definition. Service Level Objectives (SLOs) measure the service quality from the perspective of users. Unlike on-demand VMs with high availability (99.99%-99.999%) [9, 13], the spot VMSG has no SLO as spot VMs are evictable with no guarantee of availability. We provide a definition of VMSG SLO based on the availability of in-service VMs:

$$Avail = \frac{\int_0^T \min(N_{tar}, N_t^{in}) dt}{T \cdot N_{tar}}, \quad (1)$$

where N_{tar} is the target number of in-service VMs and N_t^{in} is the current number of in-service VMs at time step t , respectively, and T is the observation time window. The VMSG SLO is defined as $Avail \geq Avail_{tar}$, where $Avail_{tar}$ is given by the user, *e.g.*, three nines. Note that VMs in one VMSG have the same SKU and we focus on the number of in-service VMs instead of their resource dimensions, *e.g.*, core and memory. The extra VMs exceed N_{tar} at time step t are not included in the $Avail$ calculation.

Spot VMSG SLO. Spot VMSG could enable spot VM auto-restoration [12], *i.e.*, spot VMs are restored after evictions to meet the requirement of VM capacity. Figure 6 (a) shows the distribution of the spot VMSG Availability, excluding short-lived VMSG (less than one day).

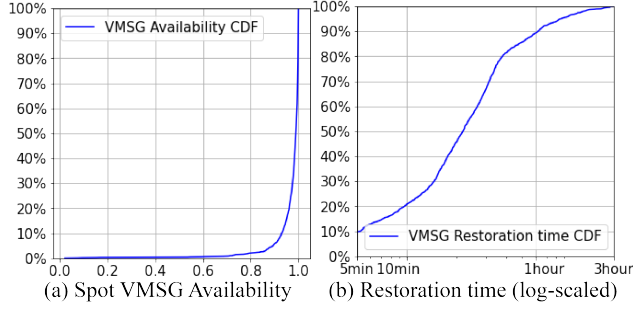


Figure 6: Distribution of the spot VMSG availability and restoration time (log-scaled).

Around 50% spot VMSG have near 99% availability with the spot restoration, still, other spot VMSGs have lower availability due to large evictions. Figure 6 (b) shows the restoration time, and 90% spot VMs could be restored within one hour. Figure 7 shows two cases where the VMSG has high and low VMSG availability due to differences in evictions and restoration time. Some spot VMs in the high availability VMSG have no evictions.

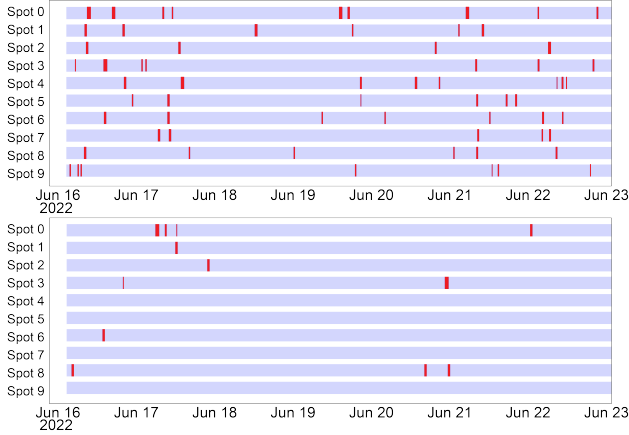


Figure 7: Two cases of low availability VMSG (upper) and high availability VMSG (lower). The red gap represents the time that spot VMs are not in service.

3.3 Spot VM Price

We investigate the spot VM price changing intervals as shown in Figure 8(a). 80% spot VMs keep the price for around one month before changing to a new price. Figure 8(b) shows the savings of using spot VMs instead of on-demand VMs, and spot VMs can achieve 60% ~ 90% savings.

3.4 High-Level Takeaways

We summarize a few insights from the above empirical study:

- The region-level eviction rate shows a temporal pattern that it is higher on peak hours, e.g., weekdays and working hours, with high resource utilization and a large amount of on-demand VM deployments. These eviction patterns show the

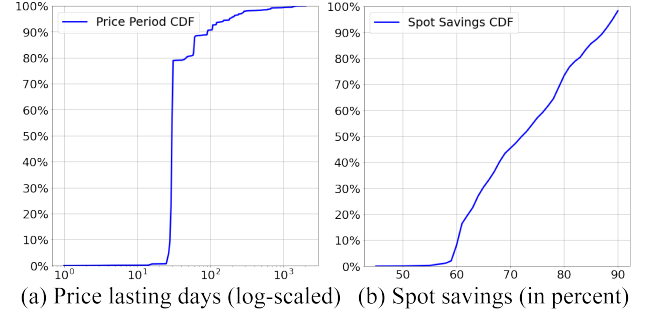


Figure 8: Distribution of spot VM price lasting period and savings compared with on-demand VM price in same SKU.

possibility of eviction prediction considering the features of time, resource utilization, and deployments.

- Spot VMs intend to have lower eviction risks if located in nodes with high spottiness and stableness. It inspires us to consider local VMSG features when learning dynamic mixture policy, including the node features where VMs in this VMSG locate.
- Around half of spot VMSGs have 90% availability which motivates us to design a mixture VMSG to improve the availability considering the large cost savings of using spot VMs.

4 THE SNAPE FRAMEWORK

In this section, we propose the Snape framework to manage a new type of VMSG that is a dynamic mixture of spot and on-demand VMs with high availability. The key is to dynamically change the number of spot and on-demand VMs based on the spot eviction estimation in order to keep the number of in-service VMs not drop below the target number.

4.1 Problem Description

When creating a VMSG with a dynamic mixture of spot and on-demand VMs, the user first specifies SLO by providing the target level of resource availability $Avail_{Tar}$ (e.g., 99.9%) for some observation time window T (e.g., one month) and a target number of VMs N_{Tar} that requires to be in-service. The mixture algorithm then determines how many spot and on-demand VMs to add or remove at each scheduled time step t . The numbers of spot and on-demand VMs to change at each time step are denoted by ΔS_t and ΔO_t (positive value for adding VMs and negative for removal), respectively. Note that adding VMs would take time to become functional while removal takes effect instantly. The decision made at each time step depends on the prices of spot and on-demand VMs, and the eviction prediction of spot VMs. The optimal decision would be achieved by minimizing the total cost over some future time window while satisfying the target SLO on availability provided by the user. Formally, one needs to solve the following optimization problem at time step t :

$$\begin{aligned} \min_{\Delta O_\tau, \Delta S_\tau} \sum_{\tau=t}^{t+j} C_\tau &= \sum_{\tau=t}^{t+j} (p_\tau^O O_\tau + p_\tau^S S_\tau) \\ \text{s.t. } Avail_\tau^P &\geq Avail_{Tar} \quad \text{for } \tau = t+1, \dots, t+j+1, \end{aligned} \quad (2)$$

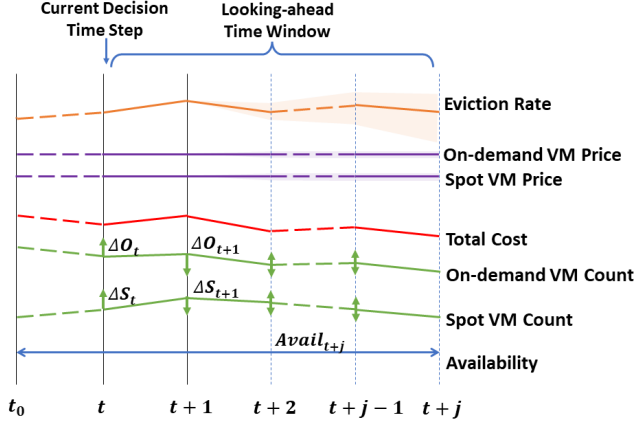


Figure 9: Illustration of decision making in mixture VMSG.

where j is the number of time step looking ahead with $j \geq 0$ and $O_t(S_t)$ denotes the total number of deployed on-demand (spot) VMs, which includes VMs that are both in service and under preparation. The track of availability over time is performed by calculating the projected availability $Avail_t^P$ within the time window $[\tau - T, \tau]$ for some future time step τ . Note that the decision variables involve the addition/removal of on-demand and spot VMs at future time steps (i.e., ΔO_{t+j} and ΔS_{t+j} for $j > 0$), however, only ΔO_t and ΔS_t are the actual action taking at time step t . Figure 9 shows a schematic diagram to illustrate the above decision-making process.

Although the above optimization approach does not work well in practice due to the prediction uncertainty and the discrepancy between the region-level eviction prediction and local VMSG evictions (discussion in Section 5), it still gives some inspiration. If the eviction rate is high in the future and will last for a while, the optimal way is to use on-demand VMs instead of spot VMs, and if this high eviction does not last long, the optimal way would be overprovisioning a certain number of spot VMs and keep fewer number of on-demand VMs. On the other hand, if the eviction rate is low in the future, the optimal way is to fully overprovision spot VMs.

4.2 The Snape Framework Overview

We demonstrate the structure of the Snape framework and the details of each module (shown in Figure 10). It manages multiple VMSGs simultaneously. The insights from empirical studies (Section 3.1) suggest (2) extracting related features from the cloud system in the global and local aspects. The global features are fed into (3) to predict region-level eviction rates in the next time steps. This global eviction prediction provides an overall estimation of the spot evictions in a long time horizon and provides suggestions for an initial mixture plan before deployments (details in Section 4.3). Moreover, global prediction is long-term and trained on long historical data, and it acts as guidance in (4) to avoid the myopic policy and accelerate learning with efficiency. In the meanwhile, (1) monitors the availability of each VMSG and checks if SLO is violated. On the other hand, the local features corresponding with VMs in the VMSG are fed into (4) to make a mixture plan with CRL (Section 4.4). The decision is made on VMs in the user's VMSG and

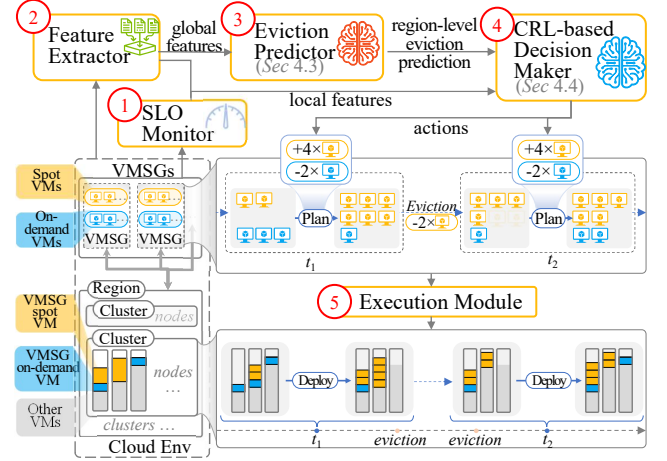


Figure 10: The overview of Snape framework.

only local features related to these VMs are required. Finally, the plan is executed by (5) to add or remove spot/on-demand VMs.

4.3 Eviction Predictor

Given the ground truth of future deployments of VMs, it is still challenging to predict spot VM evictions via simulation due to the complex cloud system dynamics. One important component of the cloud system is the allocator which assigns VMs to nodes. It is commonly associated with a large number of rules and management constraints. Thus, we adopt a data-driven approach to predict evictions that do not require a high-fidelity simulator but learn eviction behaviors from historical deployments and evictions.

The temporal eviction rate patterns in Section 3.1 require the eviction predictor to learn from long sequence time series. Recent studies have demonstrated the success of applying Transformer [55] to capture the long-range dependency. However, there are several severe issues with Transformer such as low computational efficiency and high memory usage [34, 58, 63]. We adopt an efficient Transformer, Informer [63], to predict region-level evictions by learning from long sequence monthly data. Note that we perform region-level eviction prediction instead of per-VM eviction prediction as it is challenging and costly to train a general prediction model that fits every single VM [38], and single VM eviction behaves in large variation without statistical patterns. We leverage CRL to learn the system dynamics and use the guidance of region-level prediction to make long-term decisions.

Initial mixture plan. Given the user's configuration, e.g., the target number of in-service VMs N_{Tar} , we determine an initial mixture plan based on the eviction prediction before deployment and the on-demand/spot price:

$$N_S^{Init} = \begin{cases} N_{Tar} & \text{if } \frac{1}{1-e} \leq a \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

where N_S^{Init} is the number of spot VMs in the initial plan, e is the predicted eviction rate, a is the VM price ratio $a = P_t^O / P_t^S$.

Table 1: The actions of CRL-based decision maker.

Action	Type	Example
Add	Continuous	add 4 spot VMs
Remove	Discrete	remove spot-instance2

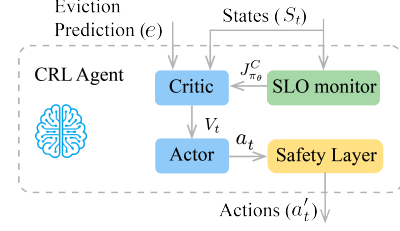
4.4 Mixture Decision Making with CRL

Given the states of VMs belonging to one VMSG, Snape's CRL-based Decision Maker (4) is designed to dynamically adjust the number of spot and on-demand VMs to minimize the overall monetary cost and retain SLOs. It makes decisions on adding or removing VMs (detailed actions in Table 1) by using the states of VMs and their corresponding node and cluster features collected from the Feature Extractor (2). The action types are different in adding and removing VMs. We do not have control over the VM allocator, and thus we cannot determine the exact nodes where new VMs will be deployed. On the other hand, we can remove specific VM instances. Snape leverages CRL to dynamically manage the number of spot and on-demand VMs to achieve long-term rewards without violations of SLOs. The high-level idea of CRL is to balance the cost and availability. For example, if the number of in-service VMs temporarily drops below the target value for a short period, the CRL will take more conservative actions in its future decision-making to ensure that SLOs are not violated.

CRL preliminary. Constrained Reinforcement Learning (CRL) is usually modeled as a Constrained Markov Decision Process (CMDP) [1], where an RL agent takes actions to maximize rewards with respect to constraints. A CMDP is represented by a tuple $(S, A, R, C, P, \mu, \gamma)$, where S is the set of states, A is the set of actions, $R : S \times A \times S \mapsto \mathbb{R}$ is the reward function, $C : S \times A \times S \mapsto \mathbb{R}$ is the cost function, $P : S \times A \times S \mapsto [0, 1]$ is the transition probability function, $\mu : S \mapsto [0, 1]$ is the initial state distribution, and γ is the discount factor for future reward. The agent's goal is to learn a policy π_θ that maximizes the discounted cumulative reward while satisfying both the cumulative constraints $J_C^{\pi_\theta}$ and instantaneous constraints $J_I^{\pi_\theta}$:

$$\begin{aligned}
 \max_{\theta} \quad & J_R^{\pi_\theta} = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right] \\
 \text{s.t.} \quad & J_C^{\pi_\theta} = \mathbb{E}_{\tau \sim \pi_\theta} \left[\frac{1}{T} \sum_{t=0}^{T-1} C(s_t, a_t, s_{t+1}) \right] \leq \epsilon \\
 & J_I^{\pi_\theta} = C(s_t, a_t, s_{t+1}) \leq \omega,
 \end{aligned} \tag{4}$$

where ϵ is the limit for cumulative constraint, ω is the limit for instantaneous constraint, and T is the total number of time steps in each trajectory. The cumulative constraint requires that the sum of the constraint variable from the beginning till the current time step is within a limit. In our problem, we set the cumulative constraint limit based on VMSG SLO, i.e., $\epsilon = 1 - Avail_{Tar}$, where the availability of in-service VMs should be above $Avail_{Tar}$. On the other hand, the instantaneous constraint is a constraint that the chosen action needs to satisfy in each step, i.e., the number of total in-service VMs should not be below the target number at each time step. Note that the fulfillment of the instantaneous constraints is a sufficient but not necessary condition for the fulfillment of the cumulative


Figure 11: CRL framework for mixture decision making.

Algorithm 1 PPO training with IPO

Input: Initialize policy π_θ with $\theta = \theta_0$ and value function V_ϕ with $\phi = \phi_0$.

Output: The policy parameter θ

- 1: **for all** iteration $k = 0, 1, 2, \dots$ **do**
 - 2: Sample N trajectories $\tau_1, \dots, \tau_N$, including observations, actions, rewards, and costs under policy π_{θ_k} .
 - 3: Compute advantage estimates $A_t^{\pi_{\theta_k}}(s_t, a_t)$ under the current value function V_{ϕ_k} , and the cumulative constraints $J_C^{\pi_{\theta_k}}$.
 - 4: Update the policy by maximizing the objective $\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} L^{IPO}(\theta)$
 - 5: Update the value function by minimizing the critic loss $\phi_{k+1} = \phi_k - \beta \nabla_{\phi} L^{critic}(\phi)$.
 - 6: **end for**
-

constraints. We include instantaneous constraints to prevent taking infeasible actions, e.g., removing all VMs at the current time step when VMSG availability is 100% in the previous time steps. As most current methods cannot guarantee zero violation for instantaneous constraints [41], we need both constraints for our problem.

CRL problem formulation. We formulate the dynamic mixture of spot and on-demand VMs problem as a sequential decision-making problem that can be solved by the above CRL framework. At each time step, (2) extracts local features corresponding to the user's VMs, including a) on the VM-level: the VM SKU, price, survived time till now, the number of spot and on-demand VMs; b) on the node-level: the state of nodes that host these VMs (node capacity, capacity utilization, spottiness, stableness); and c) on the cluster- and region-level: the aggregated capacity and capacity utilization features. Note that capacity refers to core, memory and network. The local features focus on the VMs belonging to VMSG and their co-located nodes/clusters/region. The goal of the CRL agent is to determine an optimal policy π_θ that gives the minimal monetary cost of VM utilization while keeping SLOs over the SLO time window. The action is adding or removing spot/on-demand VMs (Table 1). The reward function R is defined as the negative sum of total cost per time step, the cumulative cost is defined as the averaged sum of unavailability of in-service VMs from the start till the current time step, i.e., $J_C^{\pi_\theta} = 1 - Avail_t$, and the instantaneous cost is defined as $J_I^{\pi_\theta} = N_{Tar} - N_t^{in}$, where N_{Tar} is the target number of in-service VMs and N_t^{in} is the current number of in-service VMs at time step t . **PPO training with IPO.** Snape leverages the Proximal Policy Optimization (PPO) [50] as the base RL algorithm and Interior-point

Policy Optimization (IPO) [40] as the constraint accommodation to solve CMDP (shown in Figure 11), which achieves policy monotonic improvement and better constraint satisfaction guarantee during training. PPO is a *policy gradient* method [53] that is designed to improve training stability by limiting the changes made to update the policy at each step. It approximates the policy gradient objective with a first-order surrogate optimization approach to reduce complexity:

$$\max_{\theta} L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \eta, 1 + \eta)A_t)], \quad (5)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, A_t is the advantage function, $\text{clip}(\cdot)$ is the clip function so that $r_t(\theta)$ is clipped in $[1 - \eta, 1 + \eta]$. It helps keep the new policy close to the old policy, removing incentives for the policy to change dramatically [46]. IPO augments Equation 5 with the logarithm barrier $B(x) = \frac{\log(-x)}{t}$, where $t > 0$ is a hyper-parameter with larger t has better approximation to the indicator function. Thus, the objective function is finalized as:

$$\max_{\theta} L^{IPO}(\theta) = L^{CLIP}(\theta) + B(J_C^{\pi_{\theta}} - \epsilon), \quad (6)$$

where $J_C^{\pi_{\theta}}$ and ϵ are the cumulative constraint and limit (Equation 4). The training process follows an *actor-critic* approach [35]. The critic estimates the value function, *i.e.*, the expected cumulative discounted rewards under the current policy, and it updates by regression on the mean-squared error with reward-to-go, *i.e.*, the sum of rewards afterward:

$$\nabla L_{\phi}^{critic}(\phi) = \frac{1}{|\mathcal{D}|T} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \left(V_{\phi}(s_t) - \sum_{t'=t}^T \gamma^{t'-t} R(s_{t'}, a_{t'}, s_{t'+1}) \right) \nabla V_{\phi}(s_t), \quad (7)$$

where $\mathcal{D}$ is the set of sampled trajectories. On the other hand, the actor updates the policy by maximizing $L_{\theta}^{IPO}(\theta)$ based on the estimated value from the critic, *i.e.*, the advantage value A_t is derived from the value function by reducing bias. The pseudocode of the training algorithm is shown in Algorithm 1.

Safety Layer. IPO only deals with the cumulative constraints, and we add a safety layer [23] to the end of the policy network to satisfy instantaneous constraints $J_I^{\pi_{\theta}}$. It projects the output action to the nearest feasible action:

$$\min_{a'_t} \|a'_t - a_t\|^2, \quad \text{s.t. } J_I^{\pi_{\theta}} \leq \omega, \quad (8)$$

where a'_t is the new action from the safety layer which satisfies the instantaneous constraints. The safety layer is trying to provide a guarantee for feasible actions. For example, without the safety layer, the output action would remove all in-service VMs to lower the total cost at the final few time steps if the target number of in-service VMs is kept in all previous time steps. This aggressive action is feasible under the cumulative SLO constraints but violates the instantaneous constraints.

Prediction-guided reinforcement learning. Decision-making with long time horizons is challenging in RL which brings computational complexity and slow convergence [24, 52]. The RL agent needs a large amount of computation and interactions to make long-term spot/on-demand mixture decisions. One way to mitigate such computational and statistical issues is to regularize learning with additional knowledge, to shorten the problem horizon for fast learning [20]. We take the region-level eviction prediction results as the guidance and modify CMDP $(S, A, \tilde{R}, C, P, \mu, \tilde{\gamma})$ with a reshaped

reward and a smaller discount.

$$\tilde{R}(s_t, a_t) := R(s_t, a_t) + (1 - \lambda)\gamma \mathbb{E}[h(s_t, e)] \text{ and } \tilde{\gamma} := \lambda\gamma \quad (9)$$

where $e \in \mathbb{R}^m$ is the region-level eviction prediction on m future time horizons, h is a learnable neural network that incorporates predictions to reshape rewards.

Discrete-continuous actions. As aforementioned, the action space is discrete-continuous in our scenario. Conventional RL algorithms are not compatible with hybrid action space [37]; we then leverage the idea for hybrid control in robotics [45] where a hybrid policy π_{θ} is defined as a state-dependent distribution that jointly models discrete and continuous random variables.

$$\pi_{\theta}(a_t|s_t) = \prod_{a_t^i \in A^c} \pi_{\theta}^c(a_t^i|s_t) \prod_{a_t^i \in A^d} \pi_{\theta}^d(a_t^i|s_t), \quad (10)$$

The continuous policy π_{θ}^c is a normal distribution, and the discrete policy π_{θ}^d is a categorical distribution over K discrete choices, *i.e.*, remove certain VM instances over K VMs.

5 EVALUATION

In this section, we evaluate Snape and compare its performance with the competing strategies.

5.1 Experimental Methodology

Eviction Predictor. The training and validation dataset is three-month data of the eastern US region (the same time period as in Section 3). The data are collected per hour. At each time step, the region-level features include existing/newly-deployed spot and on-demand VMs with their related number, core/memory/network utilizations, the region capacity (core/memory/network), and the evictions at previous time steps. The input sequence includes the features of the previous one month to the current time step. The target is the region-level eviction rate, *i.e.*, the number of evicted spot VMs over the current existing number of spot VMs, after various time horizons (from 1-hour to 7-day eviction rate). We evaluate the prediction on the next month's data in the same region.

Emulator. We create an emulator as the environment for training the CRL agent. The emulator is built upon real production traces, and it maintains a VM pool for each VMSG. The CRL takes the state features from VMSG traces and Snape outputs actions on adding and removing VMs. If the action is removing specific VMs, we exclude those removed VMs from the VM pool; and if the action is adding a certain number of VMs, the emulator first checks the VMSG trace at the action time step. If the emulator cannot find a deployment of such a number of VMs in VMSG trace, it takes the VM traces with the same SKU from other users that are deployed at or close to the action time step. With the above strategy, there is no need to simulate a high-fidelity VM allocator and VM evictions.

Competing strategies. There are limited algorithms we could find to deal with mixture VMSG dynamically and most of them only set initial mixture but cannot change dynamically concerning the cloud states. We compare Snape with the following competing strategies:

- *Pure On-demand VMSG (PO):* It uses pure on-demand VMs in the VMSG.

Table 2: Eviction prediction performance with different approaches (RMSE of eviction rate prediction averaging over the granularity of 1- to 6-hour, lower is better)

Method	RMSE(%)	Method	RMSE(%)
LR [51]	42.68	Prophet [54]	18.67
RF [19]	22.69	Transformer [55]	15.23
GBDT [25]	17.77	Informer [63]	11.18
LSTM [32]	23.45	Reformer [34]	14.02
ARIMA [5]	40.80		

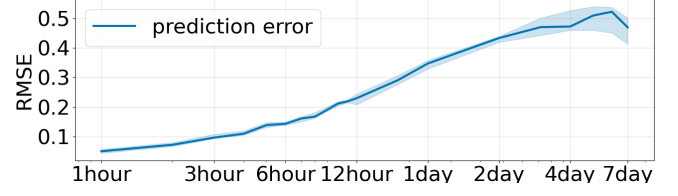
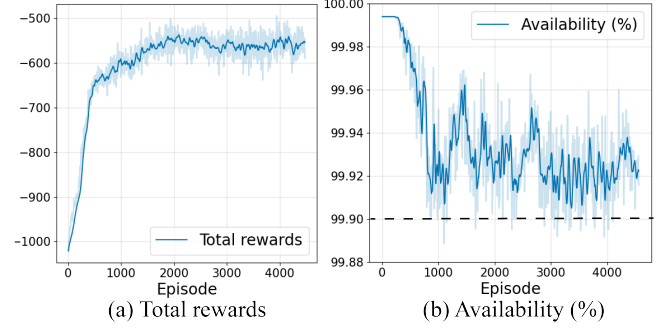
- *Pure Spot VMSG (PS)*: It uses pure spot VMSG with auto-restorations. Once spot VMs are evicted, PS tries to restart them, but no guarantee of success.
- *Overprovision Spot VMSG (OS)*: It overly adds spot VMs to replenish evicted VMs ahead of time. The overprovisioned spot VMs are based on the predicted eviction rate e , i.e., add extra $N_{tar} \cdot e / (1 - e)$ VMs. Note that we make decisions per hour, and 1-hour predicted eviction rate is used.
- *Static Mixture Ratio (SMR)*: It maintains a static spot/on-demand mixture ratio in a VMSG with auto-restoration similar in OS. The ratio is the percentage of spot VMs over on-demand VMs.
- *AutoSpotting [6] (Auto)*: It has an VMSG with a target number of on-demand VMs and it can require spot VMs to replace on-demand VMs if there is spot capacity available. As soon as one or more of these spot VMs are evicted, on-demand VMs will be created to replace evicted spot VMs.
- *Prediction-based Optimization (Opt)*: Given the predicted eviction rates in the future time horizon, it solves an Integer Linear Programming problem (ILP) to give a mixture plan (previously discussed in Section 4.1). Note that Opt outputs the number of spot/on-demand VMs to be added or removed. It cannot specify which VMs to be removed, and the removed VMs are randomly selected.

The actions of PS and SMR are eviction-triggered due to auto-restoration. Opt and OS take actions every hour. Auto's action of replacing on-demand VMs is eviction-triggered, and the action of requiring spot VMs is taken every hour.

Metrics. Two major metrics of interest are the VM cost and VMSG availability. *VM cost* is the total monetary cost that the user pays to keep their workloads running using VMSG. Note that the cost depends on the target number of VMs and the mixture of spot and on-demand VMs. In the experiments, we vary the target in-service VM number from 10-20 in a one-month time period. Then we normalize VM cost as the monetary cost per VM in one hour. *VMSG availability* measures the ability to retain SLO. If the total number of in-service VMs drops below the target number due to evictions for some time period, the SLO is violated.

5.2 Eviction Predictor

We evaluate commonly used time series prediction approaches in the industry (shown in Table 2). The transformer-based approaches have better prediction performance than regression methods (LR, RF, GBDT, Prophet, and ARIMA). It might be that the self-attention mechanism can capture a general temporal region-level pattern


Figure 12: Eviction prediction over different time horizons (log-scaled x-axis). The band represents 25th and 75th percentiles.

Figure 13: (a) Learning curve showing total reward during training, (b) Availability. The dashed line is the 99.9% SLO, and the band represents the 25th and 75th percentiles of training with 8 random seeds.

[55, 58]. It is notable that GBDT shows a close performance with transformer-based methods. Among them, Informer [63] outperforms other approaches with the ability to capture long-range dependency and temporal information. We then use Informer as the prediction approach in the eviction predictor. We evaluate eviction rate prediction on the next-month data, i.e., from July 16th to August 16th in the eastern US region. The prediction error grows exponentially with the time horizon getting longer (shown in Figure 12).

5.3 RL Training.

Figure 13(a) shows the training process that the agent was trained to get higher total rewards. At the early training stage, the CRL agent intends to overly add VMs to keep high availability, and sooner it learns to get benefits by not adding unnecessary VMs (around 300 episodes). Then it learns to leverage a proper number of spot VMs by taking the risk of eviction. Figure 13(b) shows the availability during training. The violation of the constraint, i.e., the SLO (dashed line), forces the agent to take a more conservative policy. In the end, the CRL agent converges to a policy with high total rewards while maintaining SLO.

5.4 End-to-End Performance

We evaluate Snape with its competitor baselines through experiments in Azure with metrics: *VM cost* and *VMSG availability*. The evaluation runs in a one-month time window with 20 VMSGs for each method varying the target number of in-service VMs between 10 and 20. The total VM cost is normalized to per VM cost in one hour concerning the target VM number and running time. To better

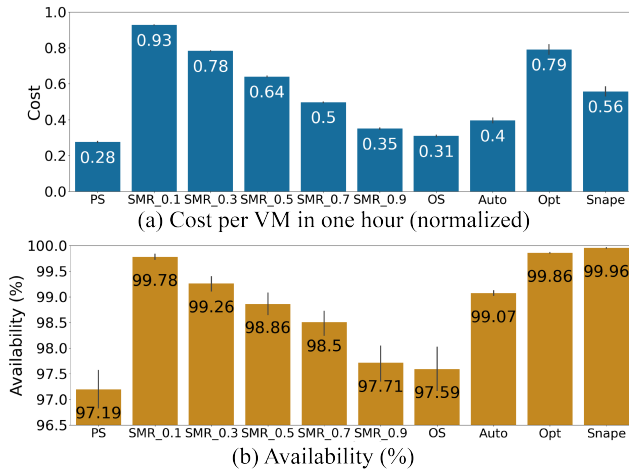


Figure 14: Snape and competitors in (a) VM cost, (b) availability. The error bar represents the 25th and 75th percentiles.

control and compare, VMs in VMSG are set to have the same SKU. We also notice that the spot and on-demand VM prices did not change dramatically during the experiment time period, *i.e.*, spot VMs are around 70% cheaper than on-demand VMs. If the ratio changes, the baselines are restarted with a new price ratio but keep the previous availability records.

As shown in Figure 14, PS has the lowest availability as the auto-restoration does not always succeed especially during peak hours. SMR with different spot VM ratios shows a cascading behavior in cost and availability. With a lower spot VM ratio, SMR has higher cost and availability. We notice that the cascading change is linear in cost but not in availability. With a large ratio of spot VMs, the availability drops dramatically with large variations. OS overprovisions spot VMs based on eviction prediction with 0.4% higher availability than PS, but the variation is large. Auto has similar availability as SMR_0.4 with a lower cost close to SMR_0.8. Auto manages to leverage on-demand VMs for time periods with high evictions and use spot VMs for the rest. It is reasonable as requiring spot capacity in high eviction time or peak hours does not always succeed which forces Auto to use on-demand VMs. Opt benefits from the eviction prediction to make optimal decisions, and it achieves higher availability than SMR_0.1 with a similar cost as SMR_0.3. However, the SLO is not achieved in Opt due to the prediction uncertainty and discrepancy between region-level eviction prediction and the local VMSG evictions. Snape keeps SLO by maintaining 99.96% availability while achieving 44% savings than using pure on-demand VMSG (PO). Snape overprovisions spot VMs when the eviction is estimated to be low. When the eviction is estimated to be high and last for a while, Snape replaces spot VMs with on-demand VMs (details will be discussed in the following case study). Note that PO is not shown in statistics as it has one unit VM cost and 100% availability.

Why does Snape outperform competitors? To demonstrate the reason behind Snape's performance over baselines, we dig deeper into case studies from online experiments and discover Snape's behaviors. Note that each VMSG runs up to one month, and we select a one-week time window and VMSG with 10 target in-service VMs.

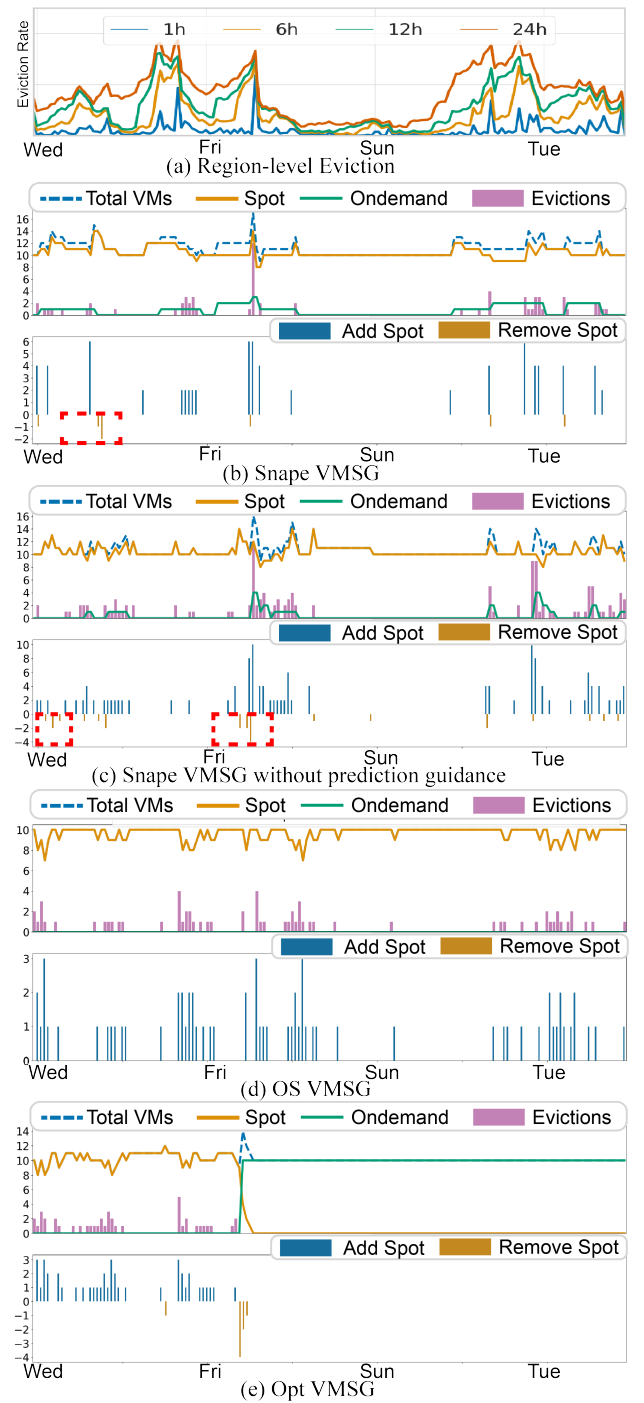


Figure 15: Case study. In each subfigure, the upper plot shows the changes in the number of total VM (dashed blue line), spot VMs (orange line) and on-demand VMs (green line), and evictions (pink bar). The lower plot shows the action of adding (blue bar) or removing (orange bar) spot VMs.

Figure 15 shows the VMSG dynamics belong to Snape and competitors in the same time window. Only OS and Opt VMSG cases are

Table 3: Status metrics of nodes hosting VMSG spot VMs.

Node status	Lucky nodes	Tragic nodes
Spottiness	0.15	-0.67
Stableness	0.72	0.24

shown, and the details of other competitors can be found in the Appendix. Snape adopts different strategies under various eviction situations.

Overprovision with discard. Snape shows similar overprovisioning behaviors as its competitor OS, *i.e.*, add spot VMs ahead of time before evictions. Interestingly, Snape proactively discards some spot VMs after overprovisioning rather than waiting for evictions (shown in Figure 15(b)(c) red dashed box). Then we perform a statistical post-analysis over the nodes that host our VMSG with node status metrics from Section 3.1, *i.e.*, *Spottiness* and *Stableness*. Table 3 shows that nodes hosting existing VMSG spot VMs (*lucky nodes*) have higher spottiness, *i.e.*, mostly collocated with other spot VMs rather than on-demand VMs, and higher stableness. The nodes that host discarded spot VMs (*tragic nodes*) are lower in both metrics. Although Snape does not have control over the VM allocator, it uses this strategy to place spot VMs on lucky nodes implicitly.

Snape leverages the region-level eviction prediction guidance. With the region-level prediction guidance, Snape estimates the severity of future evictions on VMSG and takes conservative actions by using on-demand VMs rather than only doing spot VM overprovision. On the other hand, Snape does not always keep on-demand VMs. When it has an estimation of fewer evictions in the future, it proactively replaces on-demand VMs with spot VMs. As shown in Figure 15(b), the on-demand VMs (green line) are used during the time period of high evictions, and they are removed then the future eviction is estimated to be low. In contrast, Snape without prediction guidance (Figure 15(c)) acts in a myopic manner by not keeping on-demand VMs for the high eviction time, it instead overprovisions spot VMs that results in SLO violations (round Sat and Tue).

Snape learns to estimate the VMSG evictions leveraging the local VMSG information. There is a discrepancy between the region-level evictions (Figure 15(a)) and local VMSG evictions (pink bar in the upper plot of Figure 15(b)-(e)). Only relying on the region-level eviction prediction results in ignoring the risk of evictions in the VMSG. For example, the discrepancy between the next hour region-level eviction and local VMSG eviction makes OS not overprovision enough VMs that results in SLO violations, *i.e.*, target VM number (orange line) drops below 10 in Figure 15(d). Similar trouble happens in Opt which only leverages the region-level prediction to optimize policy but ignores the local features. The discrepancy makes the unexpected evictions for Opt and SLO violations, and it turns to use on-demand VMs for the rest of the time in order to get high availability when the one-month time window is reached.

6 RELATED WORK

In recent years, there has been an increasing interest in using transient resources for a variety of workloads due to their compelling cost discount. For instance, MArk [59] and Cocktail [30] leverage spot instances to reduce the cost of serving ML models. Deepak

Narayanan et al. [44] show model training cost could be greatly reduced by optimally selecting between instance types. CompuCache [61] uses spot instances to cache in-memory data for data-intensive systems. Yanqi Zhang et al. [62] modified OpenWhisk [4] to run serverless applications over transient resources. Several work [3, 21] studied modifying and running Hadoop MapReduce over transient instances. In general, more and more applications adopt transient resources as their costs are significantly below on-demand ones. However, as pointed out by several work [3, 21, 62], the frequent interruption can degrade the application performance, which thus motivates us to offer a new type of SLO-ensured VMSG.

Spot instances are offered by all major cloud providers. Recently, researchers are proposing a new type of transient resource called harvest VMs [3, 26, 57]. Harvest VMs can leverage the CPU cores and memories of underutilized capacity as spot VMs. But unlike spot VMs with fixed sizes, harvest VMs can expand and shrink according to the available resource of the hosting node. It is shown that harvest VMs have a longer survival time compared to spot VMs [3, 62]. We note that our proposed approach can also be applied over harvest VMs. Clouds have offered scaling group [7, 16] of combined spot VMs with on-demand VMs recently. Our approach is an optimization on top of existing offerings to better trade-off between availability and cost.

Reinforcement Learning (RL) has been widely adopted to optimize system performance. For example, FIRM [48] designed an RL-based framework to optimize resource allocation for microservice applications. MA-PPO [49] used RL to optimize the resource management for serverless platforms. RL solutions have been developed for tuning database systems [60] and learning scheduling algorithms [43]. Recently, RL-based decision-making, namely Personalizer [15], has been offered by Azure as a cognitive service. Personalizer helps to make smart decisions taking the context and user reward as input. In this work, we leverage Constrained Reinforcement Learning (CRL) [27, 41] to dynamically optimize the mixture VMSG with under SLO constraints.

7 DISCUSSIONS

The Snape framework is trained and evaluated with VMSGs sharing the same VM SKU, and it could be further extended to mix VMs with different SKUs as their availability is different in peak hours and demonstrate different eviction behaviors. For example, larger core VMs have shorter lifetime (Section 3.1). The insights from Snape case studies would help adjust rules in the allocator in order to lower the probability of spot VM evictions when placing them. Another future work is to evaluate and deploy Snape to a wide range of VMSGs in production and build up a pipeline to update the CRL model in Snape in order to adapt to the environment dynamics.

8 CONCLUSION

We propose Snape, a CRL-based and prediction-guided framework to dynamically mix spot and on-demand VMs in VMSGs. It captures local VMSG information and leverages region-level eviction prediction to optimize mixture plans with good estimations of evictions. It achieves 44% savings than using on-demand VMSG while maintaining 99.96% availability.

A APPENDIX

A.1 Model Training and Update

Azure has an auto-save pipeline for data that can be easily post-processed for retraining internally. One model is trained per region and updated every two weeks. Training takes about 16 hours on a workstation equipped with AMD EPYC-7V12 64-Core CPUs, NVIDIA Tesla T4 GPU with CUDA 10.1. The training overhead is manageable given the frequency of retraining. We conducted a spearman test on the pairwise eviction rate every two weeks and observed a drop in correlation after four weeks. Further, we plan to onboard monitors to observe performance and retrain the model if necessary.

A.2 VMSG Baselines

The remaining plot of VMSG baselines is shown in Figure 16

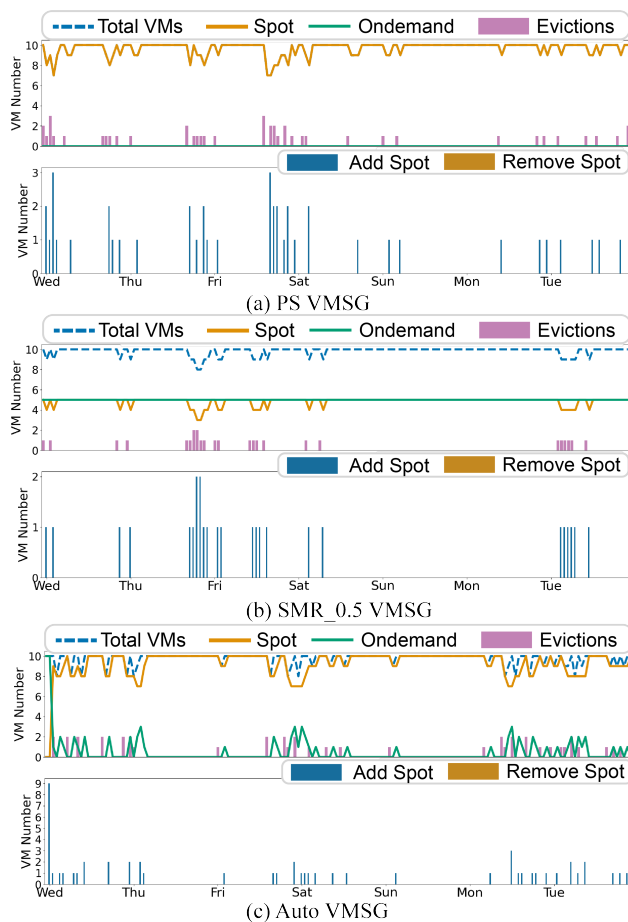


Figure 16: VMSG belongs to different baselines. (a) PS, (b) SMR_0.5, (c) Auto.

REFERENCES

- [1] Eitan Altman. 1999. *Constrained Markov decision processes: stochastic modeling*. Routledge.
- [2] Amazon. 2022. Amazon EC2 Spot Instances. <https://www.amazonaws.cn/en/ec2/spot-instances/>.
- [3] Pradeep Ambati, Íñigo Goiri, Felipe Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, Marcus Fontoura, and Ricardo Bianchini. 2020. Providing SLOs for Resource-Harvesting VMs in Cloud Platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. 735–751.
- [4] Apache. 2022. Open Source Serverless Cloud Platform. <https://openwhisk.apache.org/>. Accessed: 2022-10-19.
- [5] Adebisi A. Ariyo, Adewumi O. Adewumi, and Charles K. Ayo. 2014. Stock Price Prediction Using the ARIMA Model. In *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*. 106–112. <https://doi.org/10.1109/UKSim.2014.67>
- [6] AutoSpotting. 2022. AutoSpotting. <https://github.com/cloudutil/AutoSpotting>. Accessed: 2022-10-14.
- [7] AWS. 2022. Auto Scaling groups with multiple instance types and purchase options. <https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-mixed-instances-groups.html>.
- [8] AWS. 2022. AWS instance types. <https://docs.aws.amazon.com/AWSEC2/latest/WindowsGuide/instance-types.html>. Accessed: 2022-06-28.
- [9] AWS. 2022. AWS VM Availability. <https://docs.aws.amazon.com/whitepapers/latest/real-time-communication-on-aws/high-availability-and-scalability-on-aws.html>.
- [10] AWS. 2022. EC2 Spot Blocks. <https://aws.amazon.com/blogs/aws/new-ec2-spot-blocks-for-defined-duration-workloads/>.
- [11] Microsoft Azure. 2022. Azure Spot Virtual Machines. <https://azure.microsoft.com/en-us/services/virtual-machines/spot/#overview>.
- [12] Microsoft Azure. 2022. Azure Spot Virtual Machines: Try to restore functionality now generally available. <https://azure.microsoft.com/en-us/updates/azure-spot-virtual-machines-tryrestore-functionality-now-generally-available/>.
- [13] Microsoft Azure. 2022. Azure VM Availability. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/v1_9/.
- [14] Microsoft Azure. 2022. Azure VM SKU. <https://docs.microsoft.com/en-us/azure/virtual-machines/sizes>. Accessed: 2022-10-19.
- [15] Microsoft Azure. 2022. Personalizer. <https://learn.microsoft.com/en-us/azure/cognitive-services/personalizer/what-is-personalizer>.
- [16] Microsoft Azure. 2022. Spot Priority Mix for high availability and cost savings. <https://learn.microsoft.com/zh-cn/azure/virtual-machine-scale-sets/spot-priority-mix>. Accessed: 2022-10-19.
- [17] Microsoft Azure. 2022. Virtual Machine Scale Sets. <https://learn.microsoft.com/en-us/azure/virtual-machine-scale-sets/overview>. Accessed: 2022-09-28.
- [18] Abhinav Bhatia, Pradeep Varakantham, and Akshat Kumar. 2019. Resource constrained deep reinforcement learning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 29. 610–620.
- [19] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [20] Ching-An Cheng, Andrey Kolobov, and Adith Swaminathan. 2021. Heuristic-guided reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 13550–13563.
- [21] Navraj Chohan, Claris Castillo, Mike Spreitzer, Malgorzata Steinder, Asser Tantawi, and Chandra Krintz. 2010. See Spot Run: Using Spot Instances for Mapreduce Workflows (*HotCloud'10*).
- [22] Google Cloud. 2022. Instance groups. <https://cloud.google.com/compute/docs/instance-groups/>. Accessed: 2022-09-28.
- [23] Gal Dalal, Krishnamurthy Dvijotham, Matej Vecerik, Todd Hester, Cosmin Paduraru, and Yuval Tassa. 2018. Safe exploration in continuous action spaces. *arXiv preprint arXiv:1801.08757* (2018).
- [24] Christoph Dann and Emma Brunskill. 2015. Sample complexity of episodic fixed-horizon reinforcement learning. *Advances in Neural Information Processing Systems* 28 (2015), 2818–2826.
- [25] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.
- [26] Alex Fuerst, Stanko Novakovic, Íñigo Goiri, Gohar Irfan Chaudhry, Prateek Sharma, Kapil Arya, Kevin Broas, Eugene Bak, Mehmet Iyigun, and Ricardo Bianchini. 2022. Memory-Harvesting VMs in Cloud Platforms. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, 583–594. <https://doi.org/10.1145/3503222.3507725>
- [27] Javier Garcia and Fernando Fernández. 2015. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research* 16, 1 (2015), 1437–1480. <https://doi.org/10.5555/2789272.2886795>
- [28] Javier Garcia and Diogo Shafie. 2020. Teaching a humanoid robot to walk faster through Safe Reinforcement Learning. *Engineering Applications of Artificial Intelligence* 88 (2020), 103360. <https://doi.org/10.1016/j.engappai.2019.103360>
- [29] Google. 2022. Google Spot VMs. <https://cloud.google.com/compute/docs/instances/spot>.
- [30] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thinakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R. Das. 2022. Cocktail: A Multidimensional Optimization for Model Serving in Cloud. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*.

- [31] Ori Hadary, Luke Marshall, Ishai Menache, Abhisek Pan, Esaia E Greeff, David Dion, Star Dorminey, Shailesh Joshi, Yang Chen, Mark Russinovich, and Thomas Moscibroda. 2020. Protean: VM allocation service at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. 845–861.
- [32] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [33] Chen Hou and Qianchuan Zhao. 2017. Optimization of web service-based control system for balance between network traffic and delay. *IEEE Transactions on Automation Science and Engineering* 15, 3 (2017), 1152–1162. <https://doi.org/10.1109/TASE.2017.2746348>
- [34] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The Efficient Transformer. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26–30, 2020*. OpenReview.net.
- [35] Vijay Konda and John Tsitsiklis. 1999. Actor-critic algorithms. *Advances in neural information processing systems* 12 (1999), 1008–1014.
- [36] Iordanis Koutsopoulos and Leandros Tassioulas. 2011. Control and optimization meet the smart power grid: Scheduling of power demands for optimal energy management. In *Proceedings of the 2nd International Conference on Energy-efficient Computing and Networking*. 41–50. <https://doi.org/10.1145/2318716.2318723>
- [37] Boyan Li, Hongyao Tang, Yan Zheng, Jianye Hao, Pengyi Li, Zhen Wang, Zhaopeng Meng, and Li Wang. 2022. HyAR: Addressing Discrete-Continuous Action Reinforcement Learning via Hybrid Action Representation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net.
- [38] Benjamin Lindemann, Timo Müller, Hannes Vietz, Nasser Jazdi, and Michael Weyrich. 2021. A survey on long short-term memory networks for time series prediction. *Procedia CIRP* 99 (2021), 650–655. <https://doi.org/10.1016/j.procir.2021.03.088>
- [39] Yongshuai Liu, Jiaxin Ding, and Xin Liu. 2020. A constrained reinforcement learning based approach for network slicing. In *2020 IEEE 28th International Conference on Network Protocols (ICNP '20)*. IEEE, 1–6. <https://doi.org/10.1109/ICNP49622.2020.9259378>
- [40] Yongshuai Liu, Jiaxin Ding, and Xin Liu. 2020. IPO: Interior-point policy optimization under constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4940–4947.
- [41] Yongshuai Liu, Avishai Halev, and Xin Liu. 2021. Policy learning with constraints in model-free reinforcement learning: A survey. In *The 30th International Joint Conference on Artificial Intelligence (IJCAI)*. <https://doi.org/10.24963/ijcai.2021/614>
- [42] Xiaozhen Lu, Liang Xiao, Guohang Niu, Xiangyang Ji, and Qian Wang. 2022. Safe Exploration in Wireless Security: A Safe Reinforcement Learning Algorithm With Hierarchical Structure. *IEEE Transactions on Information Forensics and Security* 17 (2022), 732–743. <https://doi.org/10.1109/TIFS.2022.3149396>
- [43] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning Scheduling Algorithms for Data Processing Clusters (SIGCOMM '19). <https://doi.org/10.1145/3341302.3342080>
- [44] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. Analysis and Exploitation of Dynamic Pricing in the Public Cloud for ML Training. In *Workshop on Distributed Infrastructure, Systems, Programming, and AI*.
- [45] Michael Neunert, Abbas Abdolmaleki, Markus Wulfmeier, Thomas Lampe, Tobias Springenberg, Roland Hafner, Francesco Romano, Jonas Buchli, Nicolas Heess, and Martin Riedmiller. 2020. Continuous-discrete reinforcement learning for hybrid control in robotics. In *Conference on Robot Learning*. PMLR, 735–751.
- [46] OpenAI. 2022. Proximal Policy Optimization. <https://spinningup.openai.com/en/latest/algorithms/ppo.html>. Accessed: 2022-06-28.
- [47] Tu-Hoa Pham, Giovanni De Magistris, and Ryuki Tachibana. 2018. Optlayer-practical constrained optimization for deep reinforcement learning in the real world. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 6236–6243. <https://doi.org/10.1109/ICRA.2018.8460547>
- [48] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*.
- [49] Haoran Qiu, Weichao Mao, Archit Patke, Chen Wang, Hubertus Franke, Zbigniew T. Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. 2022. Reinforcement Learning for Resource Management in Multi-Tenant Serverless Platforms (EuroMLSys '22). <https://doi.org/10.1145/3517207.3526971>
- [50] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [51] George AF Seber and Alan J Lee. 2012. *Linear regression analysis*. Vol. 329. John Wiley & Sons.
- [52] Aaron Sidford, Mengdi Wang, Xian Wu, Lin Yang, and Yinyu Ye. 2018. Near-optimal time and sample complexities for solving Markov decision processes with a generative model. *Advances in Neural Information Processing Systems* 31 (2018).
- [53] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems* 12 (1999).
- [54] Taylor, Sean, J., Letham, and Benjamin. 2018. Forecasting at Scale. *American Statistician* (2018).
- [55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*. 5998–6008.
- [56] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*. 1–17. <https://doi.org/10.1145/2741948.2741964>
- [57] Yawen Wang, Kapil Arya, Marios Kogias, Manohar Vanga, Aditya Bhandari, Neeraja J. Yadwadkar, Siddhartha Sen, Sameh Elnikety, Christos Kozyrakis, and Ricardo Bianchini. 2021. SmartHarvest: Harvesting Idle CPUs Safely and Efficiently in the Cloud. In *Proceedings of the 16th European Conference on Computer Systems (EuroSys '21)*. <https://doi.org/10.1145/3447786.3456225>
- [58] Fangkai Yang, Bowen Pang, Jue Zhang, Bo Qiao, Lu Wang, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, et al. 2022. Spot Virtual Machine Eviction Prediction in Microsoft Cloud. In *Companion Proceedings of the Web Conference*. 152–156. <https://doi.org/10.1145/3487553.3524229>
- [59] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. 2019. MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. 1049–1062.
- [60] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning (SIGMOD '19). <https://doi.org/10.1145/3299869.3300085>
- [61] Qizhen Zhang, Philip Bernstein, Daniel Berger, Badrish Chandramouli, Vincent Liu, and Boon Thau Loo. 2022. Compucache: Remote computable caching using spot VMs. In *12th Conference on Innovative Data Systems Research, CIDR*. 9–12.
- [62] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. 724–739. <https://doi.org/10.1145/3477132.3483580>
- [63] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Event, February 2–9, 2021*. AAAI Press, 11106–11115.

Received 2022-10-20; accepted 2023-01-19